E2E Test Generation over a Source-Code-Derived Workflow Space: An Angular Case Study

Anonymous Author(s)

Anonymous Institution

Abstract. End-to-end (E2E) functional testing assesses whether a web application behaves correctly from the user's perspective, yet measuring such testing requires a well-defined reference space against which coverage can be computed. Existing approaches derive this space from external models, runtime exploration, or textual specifications, producing denominators that depend on the chosen abstraction or exploration budget rather than on the application's implemented behavior; as a consequence, reported coverage may overestimate adequacy by including infeasible paths or underestimate it by missing unexplored interactions. This paper proposes constructing the coverage reference space directly from frontend source code. We statically analyze Angular codebases to extract a graph of navigation structure, user-interaction sites, and the conditions (guards, parameters, visibility predicates) that determine whether each interaction is executable; from this graph, we enumerate a bounded set of workflows, each rooted at a single user trigger, classify them by feasibility, and use the result as a stable coverage denominator. Every workflow is automatically transformed into an executable browser test, and outcomes are collected with structured per-test evidence that enables systematic failure diagnosis. We evaluate the approach on seven open-source Angular applications spanning framework versions 11–20 and four authentication schemes, yielding 303 workflows with 100% test generation coverage and 70.3% execution pass rate (213 of 303 tests, median per-subject 86.5%). A systematic analysis of the 90 failing tests reveals that 71% stem from asynchronous UI rendering and permission-gate resolution at runtime rather than from deficiencies in workflow construction or test generation. The results demonstrate that a source-derived coverage denominator can separate generation completeness from execution barriers, enabling precise diagnosis of where automated E2E testing succeeds and where its current limits lie.

Keywords: End-to-End Testing · Test Coverage · Test Generation · Static Analysis · Web Application Testing · Angular · Workflow Enumeration · Failure Taxonomy

1 Introduction

End-to-end (E2E) functional testing verifies that a web application behaves correctly from the user's perspective by executing sequences of user interactions

against the running system [1]. Unlike unit or API-level testing, it exercises the full interaction stack and can reveal defects that only manifest when frontend components, routing logic, and backend services operate together [9,14,3]. When E2E testing is inadequate, user-facing defects escape to production.

The effectiveness of E2E testing depends on how thoroughly it exercises the application's interaction behavior. Coverage quantifies this thoroughness as the proportion of a defined reference space exercised by the test suite [9,30,29]; a poorly chosen reference space makes coverage misleading because high coverage over an incomplete space conceals untested behavior while low coverage over an inflated space obscures genuine adequacy. Denominator quality rests on two properties: the reference space should reflect the application's *implemented* behavior rather than an external model or a sample of explored states, and it should distinguish genuinely executable interactions from those that are structurally present but infeasible at runtime (e.g., because a guard is unsatisfied, a parameter is unbound, or a UI element is conditionally hidden).

Existing E2E approaches construct their reference spaces from external artifacts or from runtime exploration, but neither family fully addresses both properties. Static approaches [29,8,21,31,12,4,11,13] derive reference spaces from models, specifications, or LLM-generated scenarios whose fidelity to the implementation is not guaranteed. Dynamic approaches [18,19,33,1,5,26] discover behavior at runtime, but their reference spaces vary across runs, depend on the exploration budget, and typically cannot reach states behind authentication walls. To the best of our knowledge, no existing approach constructs a finite reference space of frontend interaction behavior directly from the application's source code, accounts for the executability constraints that determine which interactions are feasible, and uses the result as a stable coverage denominator for E2E test generation and evaluation.

In this paper, we address this gap by proposing an approach that derives this reference space from Angular frontend source code. Our approach statically extracts a graph of navigation structure, interaction sites, and executability constraints; enumerates a bounded set of single-trigger interaction workflows; classifies each by constraint feasibility; and uses the result as a stable denominator over which every workflow is automatically transformed into an executable Selenium test and executed with structured evidence collection.

We pursue three research questions:

RQ1. How can the set of executable frontend interactions be represented and deterministically constructed from source code as a finite, implementation-grounded reference space?

RQ2. How can every interaction in that reference space be transformed into an executable test, achieving complete generation coverage?

RQ3. What proportion of the generated tests pass against running applications, how does this compare to an exploration-based baseline, and what failure patterns emerge?

This paper makes four contributions: (i) a graph-based representation of Angular frontend interaction behavior (six node kinds, eighteen edge kinds) that

captures navigation, interaction sites, and executability constraints in a single deterministic, source-traceable artifact; (ii) an enumeration algorithm that turns this graph into a bounded reference space of constraint-classified workflows, yielding a stable coverage denominator; (iii) an automated pipeline that transforms every workflow into an executable Selenium test, collects structured per-test evidence, and computes tiered coverage over the stable denominator; and (iv) an empirical evaluation on seven open-source Angular applications showing complete test generation (303/303), 70.3% execution pass rate (213/303, median per-subject 86.5%), and a six-family residual taxonomy in which the dominant barrier (71% of failures) is runtime UI synchronization rather than generation or workflow construction. The rest of the paper reviews related work (§2), presents the approach (§3), reports the evaluation (§4), and concludes (§5).

2 Related Work

We organize related work into four families and compare them by interaction-space source, coverage semantics, and behavioral granularity.

Model-based and specification-driven testing. MBT derives test suites from behavioral models (e.g., finite state machines) using model-defined coverage criteria [29,30,8]. For web applications, models are built from specifications or artifacts such as routing configurations [20], use cases [21], requirements [7], or UML models [31]. The coverage denominator is the model or specification, whose fidelity to the implementation is not guaranteed, as guards, role constraints, and visibility conditions are typically abstracted away; granularity is at the page, state, or route level, leaving widget interactions, handler effects, and constraint surfaces unrepresented.

Source-code-based extraction. Early work by Ricca and Tonella [25] pioneered source-based navigation graph extraction and white-box test generation for web applications predating the SPA era. Jensen et al. [10] and Madsen et al. [16] show that sound static reasoning about JavaScript control and data flow is feasible, including framework-aware abstractions, but target general program properties rather than interaction-space construction. Closer to testing, JSDEP [28] computes DOM event-handler dependencies to guide the ARTEMIS platform, and Park et al. [23] analyze event-handler interactions with visibility state, though relying on dynamically collected states; both operate at event or handler granularity and do not aggregate constraint surfaces or enumerate workflows. For SPA verification, Oshima and Kishi [22] extract page-transition state machines from Angular source for model checking, and Zhang et al. [32] extract navigational graphs with guard annotations from SPA bundles for access-control verification. Both recover route-level structure but neither models widget-level interactions, computes handler-effect closures, or aggregates constraint surfaces, and neither produces executable tests or defines a coverage denominator.

Crawling and runtime exploration. Dynamic approaches construct models by executing the application: crawling [18,19], RL-guided exploration [33,27,15], GUI

ripping [17], and scriptless testing [6,24] discover states and transitions at the DOM level. The interaction space is exploration-bounded and non-deterministic, yielding variable coverage denominators across runs, and authentication walls limit exploration to unauthenticated regions [1].

LLM-based test generation and web agents. One line synthesizes test scripts from textual artifacts [12,4,11,13], deriving the interaction space from input text. A second employs LLM-driven agents. AutoE2E [1] combines crawling with feature inference, and its multimodal successor VISCA [2] adds component-level visual reasoning; other agents execute natural-language instructions [5] or reason iteratively toward targets [26]. Granularity is at the feature or task-description level. Both generation and execution are non-deterministic, and the interaction space is not grounded in source-level structure.

Positioning. Prior source-driven work shows that extraction from frontend code is feasible at multiple granularity levels [10,16,28,22,32]. We are not aware of prior work combining (i) widget-level interaction extraction from component templates and handler bodies, (ii) handler-effect closure computation, (iii) constraint-surface aggregation, (iv) finite workflow enumeration yielding a fixed coverage denominator, and (v) end-to-end realization into executable tests with integrity-verified coverage computation. Our work occupies this combination by extracting a constraint-aware workflow space from Angular source at widget-and-handler granularity, producing one executable test per workflow, and computing coverage over a fixed denominator.

3 Approach

3.1 Overview

The three research questions impose three methodological requirements: a finite, implementation-grounded, deterministic reference space of user-initiated interactions (RQ1); a transformation from every element of that space into an executable test, so that generation completeness can be evaluated independently of runtime outcomes (RQ2); and execution results reportable in a way that separates generation gaps from runtime barriers, so that a failure is attributable to the right stage (RQ3). We address these through a two-phase deterministic pipeline (Fig. 1). Phase A answers RQ1 by statically extracting a typed multi-graph from the Angular source (A1) and enumerating a bounded, constraint-classified workflow space W over it (A2). Phase B answers RQ2 and RQ3 by binding each workflow to runtime-specific values through a manifest (B0), deriving an executable action plan per workflow (B1), generating one Selenium test per plan (B2), executing every test under isolation with integrity-verified evidence (B3), and computing tiered coverage metrics that expose whether an unmet workflow failed at planning, generation, or execution (B4). Given identical source and manifest, every stage up to B3 produces byte-identical output.

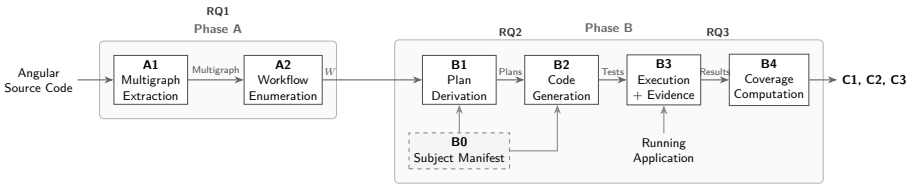

Fig. 1. Phase A extracts a deterministic multigraph and enumerates W. Phase B realizes every workflow into a Selenium test with structured observability and computes tiered coverage.

3.2 A1: UI Interaction Multigraph

The implemented interaction behavior of an Angular application is fragmented across routing configurations, component templates, handler bodies, injected services, and external navigation targets; reconstructing it as a coverage reference space requires that widget-level interactions, the routes on which they are reachable, the handler effects they trigger, and the guards and UI predicates that gate them all be captured in a single deterministic artifact amenable to bounded enumeration. A1 resolves this fragmentation by extracting a typed directed graph $G = (V, E)$—the *UI Interaction Multigraph*—whose vertex and edge types make each Angular primitive explicit and whose edges carry constraint metadata preserving executability information at the point it is observed in source.

Node kinds. V contains six kinds: **Module** (declaration scope and providers), **Route** (navigation target with path template and guards), **Component** (template owner and handler binder), **Widget** (atomic interaction site such as a button, form, or input), **Service** (injectable cross-cutting behavior), and **External** (off-application URL). Guards are treated as constraint metadata on routes and edges, not as first-class nodes.

Edge kinds. Structural edges (11 kinds) encode containment, declaration, hierarchy, and provider relationships. Executable edges (7 kinds) encode runtime transitions: four originate at widgets and cover route navigation, external navigation, handler invocation, and form submission; three originate at components or routes and cover service invocation, programmatic navigation, and redirect. Composite behavior is modelled as a set of ordered edges sharing an effect-group identifier, so the handler semantics observed in source is preserved without collapsing into a coarser abstraction. The four widget-origin kinds are the *trigger edges* enumerated by A2, and each executable edge carries a constraint surface of guard names, required route-parameter keys, and UI precondition atoms (visibility, enablement, form validity), so the feasibility information required downstream travels with the edges that witness it. A1 produces a multigraph artifact with stably sorted identifiers and source-location references on every node and edge, frozen as immutable input to downstream stages.

3.3 A2: Workflow Enumeration and Classification

A coverage denominator fit for RQ1 must be finite, deterministic, and grounded in implemented behavior, without combinatorial explosion from naive path enumeration or contamination by interactions that are structurally reachable but infeasible at runtime. A2 addresses this by rooting enumeration at user triggers rather than at paths—bounding $|W|$ by the number of enabled trigger edges and keeping the space linear in the graph's executable surface—and by classifying each workflow against the constraints aggregated along its effect closure, which excludes provably infeasible interactions without heuristics.

Augmented workflow. An *augmented workflow* w consists of a trigger edge t, an ordered effect closure $[s_1, \ldots, s_k]$ of handler-scoped service calls and navigations, an aggregated constraint surface $C(w)$, and metadata recording start route(s) and terminal node.

Trigger-centric enumeration. For each component-bearing route r (sorted by id), A2 computes the set of active components (activated by r and its ancestors, closed under composition) and the active widgets they own. A trigger edge is *enabled* if its widget is active and not unconditionally hidden or disabled. Exactly one workflow is constructed per enabled trigger; triggers belonging to components shared across several routes are aggregated over their start routes rather than duplicated, so the denominator does not inflate with the number of entry points. Algorithm 1 summarizes the procedure.

Effect closure, aggregation, and classification. When a trigger carries an effect-group identifier, A2 collects the matching service-call edges (sorted by callsite ordinal) and at most one navigation edge, then applies redirect closure with cycle detection to resolve the terminal node. The aggregated surface $C(w)$ merges per-edge surfaces by set union for guards and parameter keys and by concatenation for UI atoms. w is PRUNED when its constraints contain a provable literal contradiction, CONDITIONAL when it requires runtime assignments (guards, parameters, form validity, or expression-bound gates), and FEASIBLE otherwise; PRUNED workflows are excluded from the coverage denominator, and W is frozen before any test is generated or executed.

3.4 B0: Subject Manifest

Some executability conditions aggregated by A2—which account satisfies a given guard, what identifier binds each route parameter, how the login form of a particular deployment is actually structured—cannot be instantiated from source alone; this is a methodological boundary of purely static extraction, because the source describes the *shape* of each constraint but not its deployment-specific *instantiation*. Rather than encode subject-specific values inside W, which would conflate the source-derived denominator with environmental contingencies, B0

Algorithm 1 Augmented Workflow Enumeration and Classification

Require: Frozen multigraph $G = (V, E)$
Ensure: Workflow space W

```
W ← ∅; seen ← ∅                          ▷ seen tracks trigger edge IDs for aggregation
for all component-bearing route r ∈ G, sorted by id do
    A_c ← ActiveComponents(r)            ▷ route + ancestors + composition closure
    A_w ← ActiveWidgets(A_c)             ▷ widgets owned by active components
    for all trigger edge t from widgets in A_w, sorted by edge id do
        if t.from is statically hidden or disabled then skip
        end if
        if t.id ∈ seen then
            append r to W[t.id].startRouteIds          ▷ route aggregation
            continue
        end if
        steps ← [t]
        if t has effectGroupId then
            append CCS edges (sorted by callsite ordinal)
            append CNR edge (if present, at most one)
        end if
        terminal ← ResolveTerminal(t, steps)  ▷ redirect closure, cycle detection
        C(w) ← MergeConstraints(steps)        ▷ union of guards, params, atoms, evidence
        verdict ← Classify(C(w))              ▷ PRUNED → CONDITIONAL → FEASIBLE
        W ← W ∪ {(t.id, steps, terminal, C(w), verdict)}
        seen ← seen ∪ {t.id}
    end for
end for
return W
```

externalizes only the unresolved bindings through a per-subject manifest containing *accounts* mapping guards to credentials and roles, *route-parameter values* with optional per-template overrides, *authentication setup* (login route, form selectors, post-login success indicator), *execution configuration* for B3 (readiness endpoints, timeouts, seed and reset commands), and optional *workflow exclusions*. A CLI wizard scaffolds the manifest from A2 artifacts so that the surfaced fields correspond exactly to the unresolved constraints in W, and the completed manifest is schema-validated before B1 consumes it.

3.5 B1: Action Plan Derivation

Each $w \in W$ is symbolic—its constraints are named rather than valued, its preconditions declared rather than sequenced—so realization as a reproducible test requires both a runtime reference for the trigger widget and a sequencing of interdependent preconditions (e.g., authentication may itself require navigation; a dialog-scoped form requires the dialog to be opened first), and neither decision may alter W without invalidating its role as a fixed denominator. B1 resolves both deterministically by binding symbolic constraints to manifest val-

ues (guards to accounts, parameters to values, form fields to typed defaults), materializing preconditions in a fixed order, and selecting locators from A1 widget metadata through a priority chain that prefers stable semantic attributes over positional ones—`data-testid`, `id`, `formcontrolname`, `name`, `aria-label`, `placeholder`, static `routerlink` or `href`, scoped CSS class or tag-attribute selector, link text, and a positional fallback (`tag:nth-of-type`), each scoped to the owning component's selector when available; bindings whose values require runtime evaluation fall through to later priorities so the selector is compile-time-resolvable. B1 plans every non-skipped workflow regardless of whether its constraints are fully satisfiable, deferring any remaining failure to execution so the denominator is preserved across planning.

3.6 B2: Code Generation

Once a plan exists for every w, generation must emit a uniform executable artifact that records both the action sequence and a meaningful assertion on its outcome, yet a uniform URL-equality assertion would overclaim (many handlers do not navigate, or navigate conditionally on runtime state that static analysis cannot resolve) whereas a uniform crash-free assertion would discard signal whenever the outcome is statically determined; the tradeoff is between oracle uniformity and oracle informativeness. B2 resolves it by emitting one Selenium WebDriver test (TypeScript) per Action Plan with a fixed structure—precondition execution, action steps, and a baseline reachability postcondition chosen from three forms: URL equality when the workflow has a statically resolved terminal navigation, URL equality or unchanged with a pre-action fallback when navigation is conditional on a runtime predicate, and crash-free when no reliable URL signal is derivable. Each test embeds structured logging that records the outcome, step-level evidence (identifiers, timestamps, URL state, element resolution, screenshots), and a failure classification into a per-test execution log, which serves as the primary evidence artifact for integrity verification and diagnostic analysis.

3.7 B3: Evidence-Producing Execution

Measuring execution outcomes is meaningful only if failures can be unambiguously attributed: an unreachable application, an unresolvable locator, and a failing postcondition are methodologically distinct obstacles that must not be conflated with each other or with pipeline defects, yet a nonzero exit code alone does not distinguish a locator miss from a navigation failure, and a zero exit code does not prove that the expected state was observed. B3 combines three mutually reinforcing mechanisms to recover attribution: per-test process and browser isolation prevents inter-test contamination; a readiness check yields FAIL_APP_NOT_READY on unreachable applications, separating environmental unavailability from pipeline behavior; and integrity verification requires the persisted execution log to exist, be fresh, agree with the test-process exit code, and contain at least one step entry—any failure downgrades the outcome to

FAIL_INTEGRITY, which invalidates C_3 for the run. Outcomes fall into eight attributable categories: a PASS; six failure categories distinguishing application readiness, element location, assertion, timeout, authentication, and uncategorized errors; and a dedicated integrity-failure category that flags an untrustworthy test record.

3.8 B4: Coverage Computation

A single pass/fail number cannot indicate whether an unmet workflow failed at denominator construction, plan derivation, code generation, or runtime, so attributing coverage gaps to the correct stage is methodologically necessary rather than merely convenient. B4 therefore computes three tiers over W, with $W^* \subseteq W$ denoting executed workflows minus PRUNED outcomes, FAIL_APP_NOT_READY outcomes, and user-declared skips: C_1 and C_2 are the fractions of W with a valid plan and a syntactically valid test, and C_3 is the fraction of W^* whose outcome is PASS. The gap between C_1 and C_2 isolates generation defects and the gap between C_2 and C_3 isolates runtime barriers, so the stage at which a pipeline falls short is directly readable from the metrics. C_3 is reportable only under zero FAIL_INTEGRITY, and given frozen inputs all three metrics are deterministic.

4 Evaluation

4.1 Setup

We evaluate our approach through three evaluation questions:

- **EQ1** (RQ1): *Does Phase A produce a deterministic, auditable, bounded workflow denominator?*
- **EQ2** (RQ2): *Does Phase B achieve full generation coverage over that denominator?*
- **EQ3** (RQ3): *What execution coverage is achieved, how does it compare structurally to the strongest external approach in the same problem space, and what residual failure families delimit the current boundary?*

The replication package (pipeline, artifacts, manifests, ground truth, protocols, evaluation data) is publicly available.[1]

Subject corpus. We evaluate on seven open-source Angular applications (Table 1), drawn under three inclusion criteria: a non-trivial interaction surface (at least ten source-defined triggers), a backend provisionable locally under documented steps, and combined diversity along Angular version, authentication scheme, and backend stack. The corpus spans Angular versions 11–20, seven backend configurations (Express REST, JSON Server, Spring REST with H2, NestJS with MySQL, Spring Boot with MySQL, Express with MongoDB, and

[1] https://github.com/softscanner-sw/softscanner_fa

Table 1. Subject corpus. Columns report Angular version, authentication mechanism, A1 multigraph size (nodes/edges with structural-executable split), and A2 workflow count.

Subject	**Ng**	**Auth**	**Nodes**	**Edges**	**S/E**	$\lvert W \rvert$
posts-users-ui-ng	15	—	72	152	108/44	18
heroes-angular	11	—	67	88	63/25	19
airbus-inventory	12	JWT	68	154	101/53	21
spring-petclinic-angular	18	—	195	430	278/152	74
ever-traduora	12	JWT	253	511	376/135	109
angular-jumpstart	20	mock	109	204	121/83	47
event-booking-mean	17	bcrypt	97	196	138/58	15
Total			**861**	**1,735**	**1,185/550**	**303**

Express with static JSON), and three authentication schemes (unauthenticated, JWT, credential-based); four of the seven subjects are authentication-protected. Sizes range from 18 to 109 workflows per subject and from 67 to 253 nodes per multigraph.

Environment and execution. All runs were performed on a single workstation under Node.js 22 and headless Chrome driven by Selenium WebDriver 4. Each subject was launched locally (Docker container or local process) per its configuration requirements, and B3 waited for a readiness signal before starting execution. Manifest authoring is semi-automatic: a wizard introspects W to surface the required seed fields (credentials, route-parameter bindings, form-data overrides) and the operator supplies the subject-specific values, which took roughly 15–45 minutes per subject in our corpus. Each generated test runs in an isolated WebDriver session with per-step bounded waits (5–15 s) and a single retry on transient failure, and persists a structured execution log of step outcomes, evidence, timestamps, and the test-process exit code.

Verification procedures. The determinism protocol (EQ1) executes Phase A twice per subject from a clean state and compares the produced multigraph and workflow artifacts by SHA-256; any byte-level divergence invalidates the determinism claim. The benchmark execution protocol (EQ2–EQ3) prescribes a single B3 run per subject under the environment above, with integrity verification requiring the test-process exit code and the persisted execution log to agree for every test; a single disagreement invalidates the run. The diagnostic protocol specifies the multi-layer evidence inspection used for failure classification, in which logs are consulted first, then screenshots, and finally source-level tracing for residual cases. All metrics are computed over the workflow reference space W: C_1 is the fraction of W for which B1 produces a valid Action Plan; C_2 is the fraction for which B2 produces a syntactically valid Selenium test; C_3 is the fraction of W whose execution outcome is PASS under the integrity-verified run.

Ground truth. A ground truth (GT) set was constructed by a single auditor through systematic inspection of each subject's templates, event bindings, routing configurations, and handler bodies, cross-referenced against A1 multigraph edges and A2 workflow artifacts. For each trigger edge, the auditor verified that

the corresponding A2 workflow correctly captured trigger kind, effect closure, terminal node, and constraint surface; ambiguities were resolved by tracing the full evidence chain (source → A1 → A2). GT construction was performed by a single auditor and validates denominator correctness independently of execution outcomes; the absence of inter-rater agreement is disclosed in §4.4.

4.2 Workflow Space and Generation Coverage (EQ1–EQ2)

Phase A was executed independently twice per subject; all artifacts were byte-identical across runs (SHA-256 verified). A1 extracted 861 nodes and 1,735 edges (1,185 structural, 550 executable); A2 enumerated 303 workflows—162 FEASIBLE (53.5%), 141 CONDITIONAL (46.5%), and zero PRUNED—distributed as 186 handler triggers (61.4%), 66 route-level navigations (21.8%), 37 form submissions (12.2%), and 14 external navigations (4.6%). All enumerated workflows match the manually constructed ground truth, confirming that the denominator is complete. B1 then produced a valid Action Plan for every workflow ($C_1 = 100\%$) and B2 emitted a syntactically valid Selenium test for every plan ($C_2 = 100\%$); the 303 tests contain 483 action steps (average 1.6 per test), of which 125 (41%) carry an authentication precondition and 30 (10%) a dialog-entry precondition. Phase A extraction and enumeration complete in seconds and plan derivation and code generation (B1+B2) in under one second for the full workflow space, so the remaining coverage gap is attributable entirely to runtime execution barriers.

4.3 Execution Coverage and Residual Taxonomy (EQ3)

Execution coverage. Table 2 reports per-subject results from the integrity-verified benchmark run (0 FAIL_INTEGRITY, 303/303 logs verified). Aggregate C3 is 70.3% (213/303); median per-subject C3 is 86.5%. Four of seven subjects exceed 85%; one (angular-jumpstart) sits in the 75–85% band; the two lowest-coverage subjects are the authenticated SPAs with extensive asynchronous permission gating. Ever-traduora acts as a stress-test subject, accounting for 36% of the denominator and 63% of all failures and explaining the gap between aggregate (70.3%) and median (86.5%).

Residual taxonomy. The 90 failures were classified through the diagnostic protocol into six families (Table 3). The dominant family is *async/permission gate and repeater readiness* (64/90, 71.1%), in which the trigger widget resides behind an asynchronous permission pipe (`can: 'X' | async`), an expression-gated visibility predicate, or an `*ngFor` iteration whose data dependency does not resolve within the bounded pre-action wait; it concentrates in ever-traduora (45) and spring-petclinic-angular (9), both of which use runtime-resolved visibility extensively. Oracle-family failures—postcondition URL-match timeout (8) and external-host redirect (2)—account for 11.1% and arise from conditional navigation paths with statically indeterminate guards or from third-party redirections not resolvable from the declared target. Structural failures—locator mismatch at step time (8), step-time element materialization (7), and element-not-interactable (1)—account for the remaining 17.8%. No residual class stems from

Table 2. Execution coverage (C3) per subject. All seven subjects were executed under the benchmark protocol with zero integrity-verification disagreements.

Subject	Tests	Pass	Fail	C3
posts-users-ui-ng	18	17	1	94.4%
heroes-angular	19	17	2	89.5%
airbus-inventory	21	19	2	90.5%
spring-petclinic-angular	74	64	10	86.5%
ever-traduora	109	52	57	47.7%
angular-jumpstart	47	36	11	76.6%
event-booking-mean	15	8	7	53.3%
Total	**303**	**213**	**90**	**70.3%**

Table 3. Residual failure taxonomy: 90 failures classified into six families by diagnostic layer.

Family	Count	%	Layer
Async/permission gate or repeater readiness	64	71.1	Runtime
Postcondition URL-match timeout	8	8.9	Oracle
Locator mismatch at step time	8	8.9	Structural
Step-time element materialization	7	7.8	Structural
External-host redirect	2	2.2	Oracle
Element not interactable	1	1.1	Structural
Total	**90**	**100.0**	

an enumeration or generation defect, because all 90 residuals occur at execution while Phase B produces exactly one executable test per enumerated workflow. Failure classification relied primarily on the structured execution log; a small fraction of cases required screenshot inspection or source-level tracing.

Structural comparison with AutoE2E. We select AutoE2E [1] as the structural comparator because it is the most recent publicly inspectable end-to-end test-generation system evaluated on Angular SPAs and its subject suite shares two subjects with ours (ever-traduora and spring-petclinic-angular); crawling-based systems [19,33,27] build state-flow models rather than executable tests, and specification- or NL-driven LLM generators [11,5] require inputs unavailable for source-derived workflows without a confounding authoring step. A direct quantitative comparison is precluded by two design properties of AutoE2E: its pipeline emits a state graph and a feature database but produces no test files, assertions, or test-framework output, so no coverage metric over executable tests has a counterpart on its side; and its denominator is an LLM-inferred natural-language feature set deduplicated per run through vector-embedding similarity, which is neither finite before execution nor fixed across runs, hence not commensurable with W. We confirmed these properties empirically by executing the public AutoE2E pipeline on six applications from our corpus (including the two shared subjects) under the authors' default configuration. On spring-petclinic-angular, AutoE2E reached 9 distinguishable states, 72 actions, and 46 inferred

features before terminating at its 30-action budget; on ever-traduora, 3 states, 13 actions, and 5 features before terminating at the authentication wall; on the remaining four, the 30-action ceiling bound three runs and the authentication wall bound the fourth. Our pipeline, by contrast, enumerates 74 workflows on spring-petclinic-angular (all realized; 64 passing, 86.5%) and 109 on ever-traduora (all realized; 52 passing, 47.7%), handling authentication through manifest credential binding. The two approaches answer different questions—AutoE2E discovers features at runtime, we realize a source-derived workflow space into executable tests with deterministic oracles—so the contrast is a structural positioning, not a head-to-head metric.

Together, these results separate generation from execution as coverage bottlenecks: generation is solved relative to the source-derived denominator ($C_1 = C_2 = 100\%$), while the residual 29.7% gap at execution is dominated (71%) by runtime synchronization between the test's timing assumptions and the application's rendering pipeline, which adaptive wait strategies keyed on data-resolution state or integration with the framework's change-detection lifecycle would most directly address.

4.4 Threats to Validity

Internal validity. Execution outcomes may be affected by transient runtime conditions (browser timing, network latency, application state); we mitigate this through per-test browser isolation, bounded retry, integrity verification (zero FAIL_INTEGRITY), and pre-run stale-artifact cleanup.

Construct validity. Coverage is defined over a source-derived workflow space; this space may not capture all user-relevant behaviors (e.g., interactions involving dynamically loaded components not reachable via static analysis, or behaviors requiring multi-step stateful preconditions beyond the modelled constraint surface). The C3 postcondition oracle is a baseline reachability oracle: it verifies navigational outcome and crash-freedom but not DOM state or data correctness; a semantic oracle (C_4) is the natural next tier, left to future work. Manifest authoring is semi-automatic; although the workflow space drives the surfacing of required seed fields (credentials, route-parameter bindings, form-data overrides), the subject-specific values that cannot be inferred from source still depend on domain knowledge.

External validity. The evaluation covers seven Angular applications across framework versions 11–20, three authentication schemes, and multiple backend stacks; generalization to other frameworks or non-SPA architectures is not established. Only A1 is Angular-specific (relying on Angular template syntax, decorators, and router configuration parsing); the downstream pipeline operates over framework-agnostic JSON artifacts, so extension to React or Vue would require a framework-specific A1 extractor while reusing the remainder. Subject selection may not represent the full diversity of production Angular applications.

Conclusion validity. The aggregate C_3 (70.3%) is sensitive to subject composition, because the largest subject contributes 36% of the denominator and 63% of

the failures; we therefore report the per-subject median alongside the aggregate to mitigate this concentration. The structural rather than quantitative character of the AutoE2E comparison (see §4.3) further limits claims that can be drawn from a single external comparator.

5 Conclusion

Measuring E2E test coverage requires a well-defined reference space, yet existing approaches derive their denominators from external models or runtime exploration rather than from the application's implemented behavior. This paper demonstrates that the reference space can instead be constructed directly from frontend source code by statically extracting interaction structure and executability constraints, enumerating a bounded set of feasibility-classified workflows, and using the result as a stable coverage denominator.

Evaluation on seven Angular applications shows that the approach achieves complete test generation coverage, with all 303 source-derived workflows receiving an executable test. Of these, 70.3% pass against live applications (213 of 303; median 86.5% per subject), and the 90 failures cluster into six diagnostic families. The dominant failure cause (71.1% of all failures) is asynchronous UI rendering, in which widgets that depend on runtime-resolved data or permissions do not materialize within the test's wait window. This finding is significant because it shows that the generation problem is solved relative to the source-derived denominator; the remaining barrier is a runtime synchronization challenge between test timing and application rendering, not a deficiency in workflow construction or test planning.

Future work includes adaptive synchronization for asynchronous UI elements, stronger postcondition oracles beyond URL matching, frontend extraction for additional frameworks, and support for workflows requiring complex runtime data. The results establish that E2E coverage can be grounded in the application's implemented interaction space, providing a stable basis for comparing testing approaches independently of their exploration strategies.

References

1. Alian, P., Nashid, N., Shahbandeh, M., Shabani, T., Mesbah, A.: Feature-Driven End-to-End Test Generation. In: Proceedings of the IEEE/ACM 47th International Conference on Software Engineering. pp. 450–462. ICSE '25, IEEE Press (2025). https://doi.org/10.1109/ICSE55347.2025.00141
2. Alian, P., Tang, M., Mesbah, A.: VISCA: Inferring Component Abstractions for Automated End-to-End Testing. arXiv preprint arXiv:2506.04161 (2025)
3. Balsam, S., Mishra, D.: Web Application Testing - Challenges and Opportunities. Journal of Systems and Software **219**, 112186 (2025). https://doi.org/10.1016/j.jss.2024.112186
4. Cavalcanti, A.R., Accioly, L., Valença, G., Nogueira, S.C., Morais, A.C., Oliveira, A., Gomes, S.: Automating Test Design Using LLM: Results from an Empirical Study on the Public Sector. Conference on Digital Government Research **26** (2025). https://doi.org/10.59490/dgo.2025.1025

5. Chevrot, A., Vernotte, A., Falleri, J.R., Blanc, X., Legeard, B., Cretin, A.: Are Autonomous Web Agents Good Testers? Proceedings of the ACM on Software Engineering **2**(ISSTA), 206–228 (2025). https://doi.org/10.1145/3728879
6. Domínguez Osorio, J.M.: Automated GUI Ripping for Web Applications. Ph.D. thesis, Universidad de los Andes (2019), https://repositorio.uniandes.edu.co/handle/1992/44587
7. García, B., Dueñas, J.C.: Automated Functional Testing based on the Navigation of Web Applications. Electronic Proceedings in Theoretical Computer Science **61**, 49–65 (2011). https://doi.org/10.4204/EPTCS.61.4
8. Garousi, V., Keles, A.B., Balaman, Y., Guler, Z.O., Arcuri, A.: Model-based Testing in Practice: An Experience Report from the Web Applications Domain. Journal of Systems and Software **180**, 111032 (2021). https://doi.org/10.1016/j.jss.2021.111032
9. Garousi, V., Mesbah, A., Betin-Can, A., Mirshokraie, S.: A Systematic Mapping Study of Web Application Testing. Information and Software Technology **55**(8), 1374–1396 (2013). https://doi.org/10.1016/j.infsof.2013.02.006
10. Jensen, S.H., Madsen, M., Møller, A.: Modeling the HTML DOM and Browser API in Static Analysis of JavaScript Web Applications. In: Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering. pp. 59–69. ESEC/FSE '11, Association for Computing Machinery (2011). https://doi.org/10.1145/2025113.2025125
11. Júnior, E., Valejo, A., Valverde-Rebaza, J., Neves, V.d.O.: GenIA-E2ETest: A Generative AI-Based Approach for End-to-End Test Automation. In: Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025). pp. 282–292 (2025). https://doi.org/10.5753/sbes.2025.9927
12. Leotta, M., Yousaf, H.Z., Ricca, F., Garcia, B.: AI-Generated Test Scripts for Web E2E Testing with ChatGPT and Copilot: A Preliminary Study. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering. pp. 339–344. ACM (2024). https://doi.org/10.1145/3661167.3661192
13. Li, T., Cui, C., Huang, R., Towey, D., Ma, L.: Large Language Models for Automated Web-Form-Test Generation: An Empirical Study — RCR Report. ACM Transactions on Software Engineering and Methodology p. 3797275 (2026). https://doi.org/10.1145/3797275
14. Li, T., Huang, R., Cui, C., Towey, D., Ma, L., Li, Y.F., Xia, W.: A Survey on Web Application Testing: A Decade of Evolution (2024). https://doi.org/10.48550/arXiv.2412.10476
15. Liu, C.H., You, S.D., Chiu, Y.C.: A Reinforcement Learning Approach to Guide Web Crawler to Explore Web Applications for Improving Code Coverage. Electronics **13**(2), 427 (2024). https://doi.org/10.3390/electronics13020427
16. Madsen, M., Livshits, B., Fanning, M.: Practical static analysis of JavaScript applications in the presence of frameworks and libraries. In: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering. pp. 499–509. ESEC/FSE 2013, Association for Computing Machinery (2013). https://doi.org/10.1145/2491411.2491417
17. Memon, A., Banerjee, I., Nagarajan, A.: GUI Ripping: Reverse Engineering of Graphical User Interfaces for Testing. In: Proceedings of the 10th Working Conference on Reverse Engineering. p. 260. WCRE '03, IEEE Computer Society (2003)
18. Mesbah, A., Bozdag, E., van Deursen, A.: Crawling AJAX by Inferring User Interface State Changes. In: 2008 Eighth International Conference on Web Engineering. pp. 122–134 (2008). https://doi.org/10.1109/ICWE.2008.24

19. Mesbah, A., van Deursen, A., Lenselink, S.: Crawling Ajax-Based Web Applications through Dynamic Analysis of User Interface State Changes. ACM Trans. Web **6**(1), 3:1–3:30 (2012). https://doi.org/10.1145/2109205.2109208
20. Nass, M., Alégroth, E., Feldt, R.: Why Many Challenges with GUI Test Automation (Will) Remain. Information and Software Technology **138**, 106625 (2021). https://doi.org/10.1016/j.infsof.2021.106625
21. Nebut, C., Fleurey, F., Le Traon, Y., Jezequel, J.M.: Automatic Test Generation: A Use Case Driven Approach. IEEE Transactions on Software Engineering **32**(3), 140–155 (2006). https://doi.org/10.1109/TSE.2006.22
22. Oshima, N., Kishi, T.: Model Checking Method for SPA Page Transition Based on Component-based Framework. In: Proceedings of the 30th International Conference on Software Engineering and Knowledge Engineering (SEKE 2018). pp. 675–715 (2018). https://doi.org/10.18293/SEKE2018-038
23. Park, J., Sun, K., Ryu, S.: EventHandler-Based Analysis Framework for Web Apps Using Dynamically Collected States. In: Russo, A., Schürr, A. (eds.) Fundamental Approaches to Software Engineering. pp. 129–145. Springer International Publishing (2018). https://doi.org/10.1007/978-3-319-89363-1_8
24. Pastor Ricós, F., Slomp, A., Marín, B., Aho, P., Vos, T.E.J.: Distributed State Model Inference for Scriptless GUI Testing. Journal of Systems and Software **200**, 111645 (2023). https://doi.org/10.1016/j.jss.2023.111645
25. Ricca, F., Tonella, P.: Analysis and testing of Web applications. In: Proceedings of the 23rd International Conference on Software Engineering. pp. 25–34 (2001)
26. Shahbandeh, M., Alian, P., Nashid, N., Mesbah, A.: NaviQAte: Functionality-Guided Web Application Navigation (2024). https://doi.org/10.48550/ARXIV.2409.10741
27. Sherin, S., Muqeet, A., Khan, M.U., Iqbal, M.Z.: QExplore: An Exploration Strategy for Dynamic Web Applications Using Guided Search. Journal of Systems and Software **195**, 111512 (2023). https://doi.org/10.1016/j.jss.2022.111512
28. Sung, C., Kusano, M., Sinha, N., Wang, C.: Static DOM Event Dependency Analysis for Testing Web Applications. In: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering. pp. 447–459. FSE 2016, Association for Computing Machinery (2016). https://doi.org/10.1145/2950290.2950292
29. Utting, M., Legeard, B.: Practical Model-based Testing: A Tools Approach. Elsevier (2010)
30. Utting, M., Pretschner, A., Legeard, B.: A Taxonomy of Model-Based Testing Approaches. Software Testing, Verification and Reliability **22**(5), 297–312 (2012). https://doi.org/10.1002/stvr.456
31. Wang, C., Pastore, F., Goknil, A., Briand, L.C.: Automatic Generation of Acceptance Test Cases From Use Case Specifications: An NLP-Based Approach. IEEE Transactions on Software Engineering **48**(2), 585–616 (2022). https://doi.org/10.1109/TSE.2020.2998503
32. Zhang, T., Hao, F., Wang, Y., Zhang, B., Xie, G.: VeriFlow: A Framework for the Static Verification of Web Application Access Control via Policy-Graph Consistency. Electronics **14**(18), 3742 (2025). https://doi.org/10.3390/electronics14183742
33. Zheng, Y., Liu, Y., Xie, X., Liu, Y., Ma, L., Hao, J., Liu, Y.: Automatic Web Testing Using Curiosity-Driven Reinforcement Learning. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). pp. 423–435 (2021). https://doi.org/10.1109/ICSE43902.2021.00048